

EVENTISMA CODEBASE PDF

Complete Source Code Documentation

Categorized by Programming Languages & File Headings

Generated on: 25/07/2026, 01:26:07

MODULE INDEX & SECTIONS:

> TypeScript / React

- App.tsx (161 lines)
- types.ts (64 lines)
- initialEvents.ts (69 lines)
- Navbar.tsx (61 lines)

> Java (Spring Boot / Backend)

- EventismaApplication.java (16 lines)
- Event.java (73 lines)
- Ticket.java (64 lines)
- EventController.java (47 lines)
- TicketController.java (36 lines)
- TicketPdfService.java (44 lines)
- pom.xml (52 lines)

> CSS / Styling

- index.css (45 lines)

> JSON Config

- package.json (28 lines)
- metadata.json (6 lines)

> HTML

- index.html (12 lines)

Language: TypeScript / React (Main Application Controller)

File Path: /src/App.tsx | Lines: 161

```
import React, { useState } from 'react';
import { Navbar } from '../components/Navbar';
import { HeroBanner } from '../components/HeroBanner';
import { EventCard } from '../components/EventCard';
import { EventDetailModal } from '../components/EventDetailModal';
import { TicketCheckoutModal } from '../components/TicketCheckoutModal';
import { ScheduleTracker } from '../components/ScheduleTracker';
import { MyTicketsVault } from '../components/MyTicketsVault';
import { AddEventModal } from '../components/AddEventModal';
import { initialEvents } from '../data/initialEvents';
import { EventItem, Ticket } from '../types';
import { Filter, Sparkles, CheckCircle2, AlertCircle, Zap } from 'lucide-react';

export function App() {
  const [activeTab, setActiveTab] = useState<'explore' | 'workshops' | 'schedule' | 'tickets'>('explore');
  const [events, setEvents] = useState<EventItem[]>(initialEvents);
  const [userTickets, setUserTickets] = useState<Ticket[]>([]);
  const [searchQuery, setSearchQuery] = useState<string>('');
  const [selectedCategory, setSelectedCategory] = useState<string>('all');
  const [selectedDay, setSelectedDay] = useState<number | 'all'>('all');
  const [priceType, setPriceType] = useState<'all' | 'free' | 'paid'>('all');
  const [workshopsOnly, setWorkshopsOnly] = useState<boolean>(false);

  const [selectedEventModal, setSelectedEventModal] = useState<EventItem | null>(null);
  const [checkoutEventModal, setCheckoutEventModal] = useState<EventItem | null>(null);
  const [isAddEventOpen, setIsAddEventOpen] = useState<boolean>(false);
  const [toastMessage, setToastMessage] = useState<string | null>(null);

  const showToast = (msg: string) => {
    setToastMessage(msg);
    setTimeout(() => setToastMessage(null), 3000);
  };

  const handleTicketSuccess = (ticket: Ticket) => {
    setUserTickets((prev) => [ticket, ...prev]);
    setEvents((prev) =>
      prev.map((ev) =>
        ev.id === ticket.eventId
          ? { ...ev, registeredSlots: ev.registeredSlots + 1 }
          : ev
      )
    );
    showToast(`Pass confirmed for ${ticket.eventTitle}!`);
  };

  const handleCancelTicket = (ticketId: string) => {
    const tkt = userTickets.find((t) => t.id === ticketId);
    if (tkt) {
      setUserTickets((prev) => prev.filter((t) => t.id !== ticketId));
      setEvents((prev) =>
        prev.map((ev) =>
          ev.id === tkt.eventId
            ? { ...ev, registeredSlots: Math.max(0, ev.registeredSlots - 1) }
            : ev
        )
      );
      showToast('Ticket pass cancelled.');
```

```

if (activeTab === 'workshops' && !e.isWorkshop) return false;
if (workshopsOnly && !e.isWorkshop) return false;
if (selectedCategory !== 'all' && e.category !== selectedCategory) return false;
if (selectedDay !== 'all' && e.dayNumber !== selectedDay) return false;
if (priceType === 'free' && e.price > 0) return false;
if (priceType === 'paid' && e.price === 0) return false;

if (searchQuery.trim()) {
  const q = searchQuery.toLowerCase();
  const matchTitle = e.title.toLowerCase().includes(q);
  const matchVenue = e.venue.toLowerCase().includes(q);
  const matchSpeaker = e.speaker.name.toLowerCase().includes(q);
  const matchCat = e.category.toLowerCase().includes(q);
  return matchTitle || matchVenue || matchSpeaker || matchCat;
}

return true;
});

return (
  <div className="min-h-screen bg-[#050505] text-[#e0e0e0] font-sans">
    <Navbar
      activeTab={activeTab}
      setActiveTab={setActiveTab}
      ticketCount={userTickets.length}
      searchQuery={searchQuery}
      setSearchQuery={setSearchQuery}
      onOpenAddEvent={() => setIsAddEventOpen(true)}
    />

    {activeTab === 'explore' && (
      <HeroBanner
        onExplore={() => setActiveTab('workshops')}
        onOpenAddEvent={() => setIsAddEventOpen(true)}
      />
    )}

    </* Main Event Catalog & Views */>
    <main className="max-w-7xl mx-auto px-4 py-8">
      {activeTab === 'schedule' ? (
        <ScheduleTracker
          events={events}
          registeredEventIds={userTickets.map((t) => t.eventId)}
          onSelectEvent={(ev) => setSelectedEventModal(ev)}
          onRegisterEvent={(ev) => setCheckoutEventModal(ev)}
        />
      ) : activeTab === 'tickets' ? (
        <MyTicketsVault
          tickets={userTickets}
          onCancelTicket={handleCancelTicket}
          onExploreEvents={() => setActiveTab('explore')}
        />
      ) : (
        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
          {filteredEvents.map((ev) => (
            <EventCard
              key={ev.id}
              event={ev}
              onSelect={(e) => setSelectedEventModal(e)}
              onRegister={(e) => setCheckoutEventModal(e)}
              isUserRegistered={userTickets.some((t) => t.eventId === ev.id)}
            />
          ))}
        </div>
      )}
    </main>
  )

```

```

    { /* Modals */ }
    { selectedEventModal && (
      <EventDetailModal
        event={selectedEventModal}
        onClose={() => setSelectedEventModal(null)}
        onBookPass={(ev) => {
          setSelectedEventModal(null);
          setCheckoutEventModal(ev);
        }}
        isRegistered={userTickets.some((t) => t.eventId === selectedEventModal.id)}
      />
    )}

    { checkoutEventModal && (
      <TicketCheckoutModal
        event={checkoutEventModal}
        onClose={() => setCheckoutEventModal(null)}
        onSuccess={handleTicketSuccess}
      />
    )}

    { isAddEventOpen && (
      <AddEventModal
        onClose={() => setIsAddEventOpen(null)}
        onAddEvent={(newEvent) => {
          setEvents((prev) => [newEvent, ...prev]);
          showToast('New event created successfully!');
        }}
      />
    )}
  </div>
);
}

```

Language: TypeScript / React (Data Models & Interface Contracts)

File Path: /src/types.ts | Lines: 64

```
export type EventType = 'workshop' | 'hackathon' | 'keynote' | 'cultural' | 'competition' | 'esports';
export type PassTier = 'General Pass' | 'VIP Front-Row' | 'Workshop Bundle';

export interface AgendaItem {
  time: string;
  activity: string;
  speaker?: string;
}

export interface Speaker {
  name: string;
  role: string;
  avatar: string;
  org?: string;
}

export interface EventItem {
  id: string;
  title: string;
  subtitle: string;
  type: EventType;
  category: string;
  description: string;
  fullAgenda: AgendaItem[];
  date: string;
  startTime: string;
  endTime: string;
  dayNumber: number;
  venue: string;
  speaker: Speaker;
  price: number;
  totalSlots: number;
  registeredSlots: number;
  bannerGradient: string;
  bannerImage: string;
  tags: string[];
  prerequisites: string[];
  perks: string[];
  featured?: boolean;
  isWorkshop?: boolean;
  organizer?: string;
}

export interface Ticket {
  id: string;
  ticketHash: string;
  eventId: string;
  eventTitle: string;
  eventDate: string;
  eventTime: string;
  venue: string;
  passType: PassTier;
  studentName: string;
  studentId: string;
  department: string;
  email: string;
  phone: string;
  pricePaid: number;
  paymentMethod: string;
  purchasedAt: string;
  seatZone: string;
}
```


Language: TypeScript / React (Initial Festival Event Catalog)

File Path: /src/data/initialEvents.ts | Lines: 69

```
import { EventItem } from '../types';

export const initialEvents: EventItem[] = [
  {
    id: 'evt-001',
    title: 'Generative AI & LLM Systems Masterclass',
    subtitle: 'Hands-on transformer architectures, prompt engineering, and Gemini API integration.',
    type: 'workshop',
    category: 'Artificial Intelligence',
    description: 'Deep dive into state-of-the-art Generative AI pipelines and fine-tuning lightweight LLM',
    fullAgenda: [
      { time: '10:00 AM', activity: 'Intro to Transformer Neural Networks', speaker: 'Dr. Elena Rostova' },
      { time: '11:15 AM', activity: 'Building Multimodal Agents with Gemini SDK', speaker: 'Dr. Elena Ros' },
      { time: '12:30 PM', activity: 'Hands-on Hack Lab & Code Deployment', speaker: 'Dev Team' }
    ],
    date: 'AUG 14, 2026',
    startTime: '10:00 AM',
    endTime: '01:00 PM',
    dayNumber: 1,
    venue: 'Cyber Auditorium - Hall A',
    speaker: {
      name: 'Dr. Elena Rostova',
      role: 'Chief AI Scientist, DeepLab',
      avatar: 'https://images.unsplash.com/photo-1573496359142-b8d87734a5a2?w=150&auto=format&fit=crop&q=80'
    },
    price: 0,
    totalSlots: 150,
    registeredSlots: 128,
    bannerGradient: 'from-cyan-500 via-blue-600 to-indigo-900',
    bannerImage: 'https://images.unsplash.com/photo-1618005182384-a83a8bd57fbe?w=800&auto=format&fit=crop&q=80',
    tags: ['AI', 'Python', 'Gemini', 'LLM'],
    prerequisites: ['Basic Python or TypeScript knowledge'],
    perks: ['Verified AI Certificate', 'Gemini API Credits', 'Swag Kit'],
    featured: true,
    isWorkshop: true
  },
  {
    id: 'evt-002',
    title: 'Autonomous Robotics & Vision Workshop',
    subtitle: 'Program ROS 2 robots with computer vision and real-time obstacle avoidance.',
    type: 'workshop',
    category: 'Robotics',
    description: 'Construct and program hardware vision systems using LiDAR and OpenCV on embedded Raspbe',
    fullAgenda: [
      { time: '02:00 PM', activity: 'LiDAR Mapping & Sensor Fusion', speaker: 'Prof. Kenji Sato' },
      { time: '03:30 PM', activity: 'Autonomous Navigation Circuit Challenge', speaker: 'Robotics Guild' }
    ],
    date: 'AUG 15, 2026',
    startTime: '02:00 PM',
    endTime: '05:00 PM',
    dayNumber: 2,
    venue: 'Mechatronics Innovation Arena',
    speaker: {
      name: 'Prof. Kenji Sato',
      role: 'Director, CyberRobotics Lab',
      avatar: 'https://images.unsplash.com/photo-1534528741775-53994a69daeb?w=150&auto=format&fit=crop&q=80'
    },
    price: 15,
    totalSlots: 80,
    registeredSlots: 74,
    bannerGradient: 'from-purple-600 via-pink-600 to-rose-900',
    bannerImage: 'https://images.unsplash.com/photo-1534528741775-53994a69daeb?w=800&auto=format&fit=crop&q=80'
  }
];
```

```
bannerImage: 'https://images.unsplash.com/photo-1485827404703-89b55fcc595e?w=800&auto=format&fit=crop'
tags: ['ROS2', 'Computer Vision', 'Embedded'],
prerequisites: ['C++ or Python Basics'],
perks: ['Hardware Micro-Controller Kit', 'Printed Workbook'],
featured: true,
isWorkshop: true
}
];
```


Language: TypeScript / React (Navigation Bar)

File Path: /src/components/Navbar.tsx | Lines: 61

```
import React from 'react';
import { Sparkles, Calendar, Ticket, PlusCircle, Search, Layers, Zap, Code2 } from 'lucide-react';

interface NavbarProps {
  activeTab: 'explore' | 'workshops' | 'schedule' | 'tickets';
  setActiveTab: (tab: 'explore' | 'workshops' | 'schedule' | 'tickets') => void;
  ticketCount: number;
  searchQuery: string;
  setSearchQuery: (query: string) => void;
  onOpenAddEvent: () => void;
  onOpenCodeExport: () => void;
}

export const Navbar: React.FC<NavbarProps> = ({
  activeTab,
  setActiveTab,
  ticketCount,
  searchQuery,
  setSearchQuery,
  onOpenAddEvent,
  onOpenCodeExport,
}) => {
  return (
    <header className="sticky top-0 z-40 bg-[#0a0a0c]/90 backdrop-blur-md border-b border-[#1e1e24] px-4">
      <div className="max-w-7xl mx-auto flex items-center justify-between gap-4">
        <div className="flex items-center gap-3 cursor-pointer" onClick={() => setActiveTab('explore')}>
          <div className="w-8 h-8 bg-[#00f3ff] rounded-sm shadow-[0_0_15px_#00f3ff] flex items-center justify-center">
            <Zap className="w-5 h-5 fill-black stroke-black" />
          </div>
          <div>
            <span className="font-extrabold text-xl text-white font-mono">
              EVENT<span className="text-[#00f3ff]">ISMA</span>
            </span>
          </div>
        </div>
        <div>
          <span className="font-extrabold text-xl text-white font-mono">
            EVENT<span className="text-[#00f3ff]">ISMA</span>
          </span>
        </div>
      </div>

      <nav className="flex items-center gap-2">
        <button onClick={() => setActiveTab('explore')} className="px-3 py-1.5 text-xs text-gray-300 font-medium">
          DASHBOARD
        </button>
        <button onClick={() => setActiveTab('workshops')} className="px-3 py-1.5 text-xs text-gray-300 font-medium">
          WORKSHOPS
        </button>
        <button onClick={() => setActiveTab('schedule')} className="px-3 py-1.5 text-xs text-gray-300 font-medium">
          MY SCHEDULE
        </button>
        <button onClick={() => setActiveTab('tickets')} className="px-3 py-1.5 text-xs text-gray-300 font-medium">
          WALLET ({ticketCount})
        </button>
        <button
          onClick={onOpenCodeExport}
          className="flex items-center gap-1.5 px-3 py-1.5 bg-[#111116] border border-[#00f3ff] text-[#00f3ff] font-medium"
        >
          <Code2 className="w-4 h-4" />
          PDF CODE
        </button>
      </nav>
    </div>
  </header>
  </div>
);
};
```

Language: Java (Spring Boot Application Launcher)

File Path: /src/main/java/com/eventisma/EventismaApplication.java | Lines: 16

```
package com.eventisma;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class EventismaApplication {

    public static void main(String[] args) {
        SpringApplication.run(EventismaApplication.class, args);
        System.out.println("=====");
        System.out.println("    EVENTISMA JAVA BACKEND SERVER RUNNING  ");
        System.out.println("    REST API: http://localhost:8080/api    ");
        System.out.println("=====");
    }
}
```

Language: Java (JPA Domain Model for Event & Workshops)

File Path: /src/main/java/com/eventisma/model/Event.java | Lines: 73

```
package com.eventisma.model;

import jakarta.persistence.*;
import java.util.List;

@Entity
@Table(name = "events")
public class Event {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private String id;

    private String title;
    private String subtitle;
    private String type;
    private String category;

    @Column(length = 2000)
    private String description;

    private String date;
    private String startTime;
    private String endTime;
    private int dayNumber;
    private String venue;
    private double price;
    private int totalSlots;
    private int registeredSlots;
    private boolean isWorkshop;
    private boolean featured;

    @ElementCollection
    private List<String> tags;

    @ElementCollection
    private List<String> perks;

    // Constructors
    public Event() {}

    public Event(String title, String subtitle, String category, double price, int totalSlots, String ven
        this.title = title;
        this.subtitle = subtitle;
        this.category = category;
        this.price = price;
        this.totalSlots = totalSlots;
        this.venue = venue;
        this.registeredSlots = 0;
    }

    // Getters and Setters
    public String getId() { return id; }
    public void setId(String id) { this.id = id; }

    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }

    public String getCategory() { return category; }
    public void setCategory(String category) { this.category = category; }
```

```
public double getPrice() { return price; }
public void setPrice(double price) { this.price = price; }

public int getTotalSlots() { return totalSlots; }
public void setTotalSlots(int totalSlots) { this.totalSlots = totalSlots; }

public int getRegisteredSlots() { return registeredSlots; }
public void setRegisteredSlots(int registeredSlots) { this.registeredSlots = registeredSlots; }

public boolean isWorkshop() { return isWorkshop; }
public void setWorkshop(boolean isWorkshop) { this.isWorkshop = isWorkshop; }
}
```

Language: Java (JPA Domain Model for Digital Ticket Pass)

File Path: /src/main/java/com/eventisma/model/Ticket.java | Lines: 64

```
package com.eventisma.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "tickets")
public class Ticket {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private String id;

    @Column(unique = true, nullable = false)
    private String ticketHash;

    private String eventId;
    private String eventTitle;
    private String eventDate;
    private String eventTime;
    private String venue;
    private String passType;

    private String studentName;
    private String studentId;
    private String department;
    private String email;
    private String phone;

    private double pricePaid;
    private String paymentMethod;
    private LocalDateTime purchasedAt;
    private String seatZone;
    private String qrData;

    public Ticket() {
        this.purchasedAt = LocalDateTime.now();
    }

    // Business Method to Generate Security Hash
    public void generateHash() {
        this.ticketHash = "EVT-" + System.currentTimeMillis() + "-" + Math.abs(studentId.hashCode());
        this.qrData = "VERIFIED_EVENTISMA_PASS:" + this.ticketHash + ":" + this.studentId;
    }

    // Getters and Setters
    public String getId() { return id; }
    public void setId(String id) { this.id = id; }

    public String getTicketHash() { return ticketHash; }
    public void setTicketHash(String ticketHash) { this.ticketHash = ticketHash; }

    public String getEventTitle() { return eventTitle; }
    public void setEventTitle(String eventTitle) { this.eventTitle = eventTitle; }

    public String getStudentName() { return studentName; }
    public void setStudentName(String studentName) { this.studentName = studentName; }

    public String getStudentId() { return studentId; }
    public void setStudentId(String studentId) { this.studentId = studentId; }
```


Language: Java (Spring REST API Controller - Event Management)

File Path: /src/main/java/com/eventisma/controller/EventController.java | Lines: 47

```
package com.eventisma.controller;

import com.eventisma.model.Event;
import com.eventisma.repository.EventRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/events")
@CrossOrigin(origins = "**")
public class EventController {

    @Autowired
    private EventRepository eventRepository;

    @GetMapping
    public ResponseEntity<List<Event>> getAllEvents(
        @RequestParam(required = false) String category,
        @RequestParam(required = false) Boolean workshopOnly) {

        List<Event> events;
        if (workshopOnly != null && workshopOnly) {
            events = eventRepository.findByIsWorkshopTrue();
        } else if (category != null && !category.equals("all")) {
            events = eventRepository.findByCategoryIgnoreCase(category);
        } else {
            events = eventRepository.findAll();
        }
        return ResponseEntity.ok(events);
    }

    @GetMapping("/{id}")
    public ResponseEntity<Event> getEventById(@PathVariable String id) {
        return eventRepository.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    public ResponseEntity<Event> createEvent(@RequestBody Event event) {
        Event saved = eventRepository.save(event);
        return ResponseEntity.ok(saved);
    }
}
```

Language: Java (Spring REST API Controller - Pass Booking)

File Path: /src/main/java/com/eventisma/controller/TicketController.java | Lines: 36

```
package com.eventisma.controller;

import com.eventisma.model.Ticket;
import com.eventisma.service.TicketPdfService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/tickets")
@CrossOrigin(origins = "")
public class TicketController {

    @Autowired
    private TicketPdfService ticketPdfService;

    @PostMapping("/checkout")
    public ResponseEntity<Ticket> bookPass(@RequestBody Ticket ticketRequest) {
        ticketRequest.generateHash();
        return ResponseEntity.ok(ticketRequest);
    }

    @GetMapping("/{ticketId}/pdf")
    public ResponseEntity<byte[]> downloadTicketPdf(@PathVariable String ticketId) {
        byte[] pdfBytes = ticketPdfService.generateTicketPdf(ticketId);

        return ResponseEntity.ok()
            .header(HttpHeaders.CONTENT_DISPOSITION, "attachment; filename=Eventisma_Pass_" + ticketId)
            .contentType(MediaType.APPLICATION_PDF)
            .body(pdfBytes);
    }
}
```


Language: Java (iText Document Generator for PDF Passes)

File Path: /src/main/java/com/eventisma/service/TicketPdfService.java | Lines: 44

```
package com.eventisma.service;

import com.itextpdf.text.*;
import com.itextpdf.text.pdf.*;
import org.springframework.stereotype.Service;

import java.io.ByteArrayOutputStream;

@Service
public class TicketPdfService {

    public byte[] generateTicketPdf(String ticketId) {
        ByteArrayOutputStream out = new ByteArrayOutputStream();
        try {
            Document document = new Document(PageSize.A4);
            PdfWriter.getInstance(document, out);
            document.open();

            // Document Header
            Font titleFont = FontFactory.getFont(FontFactory.HELVETICA_BOLD, 22, BaseColor.CYAN);
            Paragraph title = new Paragraph("EVENTISMA // DIGITAL TICKET PASS", titleFont);
            title.setAlignment(Element.ALIGN_CENTER);
            document.add(title);

            document.add(new Paragraph(" ")); // Spacer

            // Pass Details Table
            PdfPTable table = new PdfPTable(2);
            table.setWidthPercentage(100);
            table.addCell("Ticket Security Hash:");
            table.addCell("EVT-" + ticketId.toUpperCase());
            table.addCell("Access Tier:");
            table.addCell("VIP Front-Row / Workshop Pass");
            table.addCell("Status:");
            table.addCell("CONFIRMED & VERIFIED");

            document.add(table);
            document.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
        return out.toByteArray();
    }
}
```

Language: Java (Maven Project Metadata & Dependencies)

File Path: /pom.xml | Lines: 52

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2000/svg-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd"
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.eventisma</groupId>
  <artifactId>eventisma-backend</artifactId>
  <version>1.0.0-SNAPSHOT</version>
  <name>Eventisma Java Platform</name>
  <description>Enterprise Java Spring Boot REST API for Eventisma Festival Matrix</description>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.2</version>
  </parent>

  <properties>
    <java.version>17</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>com.h2database</groupId>
      <artifactId>h2</artifactId>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>com.itextpdf</groupId>
      <artifactId>itextpdf</artifactId>
      <version>5.5.13.3</version>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

Language: CSS / Styling (Tailwind Base & Cyber Utilities)

File Path: /src/index.css | Lines: 45

```
@import "tailwindcss";

@layer base {
  body {
    background-color: #050505;
    color: #e0e0e0;
    font-family: system-ui, -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
    overflow-x: hidden;
  }
}

.cyber-grid-bg {
  background-image:
    linear-gradient(to right, rgba(0, 243, 255, 0.04) 1px, transparent 1px),
    linear-gradient(to bottom, rgba(0, 243, 255, 0.04) 1px, transparent 1px);
  background-size: 32px 32px;
}

.neon-glow-cyan {
  box-shadow: 0 0 15px rgba(0, 243, 255, 0.35), inset 0 0 10px rgba(0, 243, 255, 0.1);
}

.clip-cyber-corner {
  clip-path: polygon(
    0 0,
    calc(100% - 12px) 0,
    100% 12px,
    100% 100%,
    12px 100%,
    0 calc(100% - 12px)
  );
}

@media print {
  body {
    background: white !important;
    color: black !important;
  }
  .no-print {
    display: none !important;
  }
  .print-only {
    display: block !important;
  }
}
```

Language: JSON Configuration (Project Dependencies & Scripts)

File Path: /package.json | Lines: 28

```
{
  "name": "eventisma-fest-app",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite --port=3000 --host=0.0.0.0",
    "build": "vite build",
    "preview": "vite preview",
    "lint": "tsc --noEmit"
  },
  "dependencies": {
    "@google/genai": "^2.4.0",
    "canvas-confetti": "^1.9.4",
    "express": "^4.21.2",
    "jspdf": "^2.5.2",
    "lucide-react": "^0.546.0",
    "motion": "^12.23.24",
    "react": "^19.0.1",
    "react-dom": "^19.0.1",
    "vite": "^6.2.3"
  },
  "devDependencies": {
    "@types/node": "^22.14.0",
    "tailwindcss": "^4.1.14",
    "typescript": "~5.8.2"
  }
}
```

Language: JSON Configuration (App Manifest)

File Path: /metadata.json | Lines: 6

```
{  
  "name": "Eventisma",  
  "description": "Futuristic college event management platform with neon dark aesthetic, workshop registr  
  "requestFramePermissions": [],  
  "majorCapabilities": ["MAJOR_CAPABILITY_SERVER_SIDE_GEMINI_API"]  
}
```

Language: HTML (Web Application Entry Point)

File Path: /index.html | Lines: 12

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Eventisma - Festival Matrix</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```